

Εργαστήριο 1 – Εισαγωγή στις Κλάσεις: Protein και Gene

Πλήρης Θεωρητικός Οδηγός

ΠΕΡΙΛΗΨΗ ΑΣΚΗΣΗΣ

Τι Ζητάει η Άσκηση

Η άσκηση σας ζητάει να δημιουργήσετε:

1. Κλάση Protein με:

- Πεδία: `id`, `name`, `sequence` (αλληλουχία αμινοξέων)
- Μεθόδους `set` και `get` για κάθε πεδίο
- Μέθοδο `describe()` ΠΟΥ ΕΚΤΥΠΩΝΕΙ: `"protein [ονομα] [αλληλουχία]"`
- Προαιρετικά: μέθοδο `length()` για το μήκος της αλληλουχίας

2. Κλάση Gene με:

- Πεδία: `id`, `name`, `chrom`, `start`, `end`, `strand`
- Μεθόδους `set` και `get` για κάθε πεδίο
- Μέθοδο `describe()` ΠΟΥ ΕΚΤΥΠΩΝΕΙ: `"Gene [ονομα] (chr[chrom]:[start]-[end], [strand] strand)"`

3. Κύριο πρόγραμμα

- Δημιουργεί **τουλάχιστον 2 αντικείμενα** κάθε τύπου
 - Εκτυπώνει τις περιγραφές τους
-

ΑΝΑΛΥΤΙΚΗ ΕΞΗΓΗΣΗ ΟΡΙΣΜΩΝ

1. Κλάση (Class)

Ορισμός:

Ένα "καλούπι" ή "σχέδιο" για τη δημιουργία αντικειμένων. Ορίζει τη **δομή** (πεδία/attributes) και τη **συμπεριφορά** (μεθόδους/methods) των αντικειμένων.

Σύνταξη:

```
class ΟνομαΚλάσης {  
    private:  
        // Ιδιωτικά πεδία  
  
    public:  
        // Δημόσιες μέθοδοι  
};
```

Παράδειγμα:

```
class Protein {  
    // Περιεχόμενο κλάσης  
};
```

Αναλογία:

Η κλάση είναι σαν το αρχιτεκτονικό σχέδιο ενός σπιτιού - δείχνει πώς θα φτιαχτεί, αλλά δεν είναι το ίδιο το σπίτι.

2. Αντικείμενο (Object)

Ορισμός:

Μια **συγκεκριμένη περίπτωση** (instance) μιας κλάσης. Είναι το πραγματικό "προϊόν" που δημιουργείται από το "καλούπι" της κλάσης.

Σύνταξη:

```
ΟνομαΚλάσης ονομαΑντικειμενου;
```

Παράδειγμα:

```
Protein protein1; // Δημιουργία αντικειμένου  
Protein protein2; // Άλλο αντικείμενο της ίδιας κλάσης
```

Αναλογία:

Αν η κλάση είναι το σχέδιο του σπιτιού, το αντικείμενο είναι το πραγματικό σπίτι που χτίστηκε.

3. Πεδία (Fields / Attributes / Member Variables)

Ορισμός:

Οι **μεταβλητές** που ανήκουν στην κλάση και αποθηκεύουν τα δεδομένα του αντικειμένου.

Παράδειγμα:

```
private:  
    int id;        // Πεδίο τύπου integer  
    string name;   // Πεδίο τύπου string  
    string sequence; // Πεδίο τύπου string
```

Χαρακτηριστικά:

- Κάθε αντικείμενο έχει τα **δικά του** πεδία
- Συνήθως είναι `private` για προστασία

4. Μέθοδοι (Methods / Member Functions)

Ορισμός:

Οι **συναρτήσεις** που ανήκουν σε μια κλάση και εκτελούν λειτουργίες πάνω στα δεδομένα του αντικειμένου.

Παράδειγμα:

```
void describe() {  
    cout << "protein " << this->name << " " << this->sequence << endl;  
}
```

Χαρακτηριστικά:

- Έχουν πρόσβαση στα πεδία της κλάσης
- Συνήθως είναι `public` για να μπορούν να κληθούν από έξω

5. Encapsulation (Εγκλωβισμός / Ενθυλάκωση)

Ορισμός:

Η αρχή της **απόκρυψης** των εσωτερικών δεδομένων μιας κλάσης. Τα πεδία είναι `private` και προσπελαύνονται μόνο μέσω `public` μεθόδων.

Πλεονεκτήματα:

1. **Προστασία δεδομένων:** Αποτροπή τυχαίων ή κακόβουλων αλλαγών
2. **Έλεγχος πρόσβασης:** Μπορούμε να ελέγξουμε τι τιμές επιτρέπονται
3. **Ευκολία συντήρησης:** Αλλαγές στα εσωτερικά δεν επηρεάζουν τον εξωτερικό κώδικα
4. **Ευελιξία:** Μπορούμε να αλλάξουμε την εσωτερική υλοποίηση χωρίς να επηρεάσουμε το interface

Παράδειγμα:

```
class Protein {  
    private:  
        int id; // Κρυφό - δεν μπορεί να προσπελαστεί απευθείας  
  
    public:  
        void setId(int newId) { // Ελεγχόμενη πρόσβαση  
            if (newId > 0) {  
                this->id = newId;  
            }  
        }  
};
```

Αναλογία:

Σαν το ATM της τράπεζας - δεν έχετε απευθείας πρόσβαση στα χρήματα, αλλά μέσω συγκεκριμένων λειτουργιών (κατάθεση, ανάληψη).

6. Setters (Μέθοδοι Ορισμού / Mutators)

Ορισμός:

Μέθοδοι που **θέτουν** (set) τιμές στα ιδιωτικά πεδία της κλάσης.

Σύνταξη:

```
void setΌνομαΠεδίου(ΤύποςΔεδομένων παράμετρος) {  
    this->πεδίο = παράμετρος;  
}
```

Παράδειγμα:

```
void setName(string newName) {  
    this->name = newName;  
}  
  
void setId(int newId) {  
    if (newId > 0) { // Μπορούμε να προσθέσουμε έλεγχο  
        this->id = newId;  
    }  
}
```

Χρήση:

```
protein1.setName("PTEN");  
protein1.setId(1);
```

Χαρακτηριστικά:

- Return type: συνήθως `void`
- Ονομασία: συνήθως `setXXX`
- Μπορούν να περιλαμβάνουν έλεγχο εγκυρότητας

7. Getters (Μέθοδοι Ανάγνωσης / Accessors)

Ορισμός:

Μέθοδοι που **επιστρέφουν** (get) τιμές από τα ιδιωτικά πεδία της κλάσης.

Σύνταξη:

```
ΤύποςΔεδομένων getΌνομαΠεδίου() {  
    return this→πεδίο;  
}
```

Παράδειγμα:

```
string getName() {  
    return this→name;  
}  
  
int getId() {  
    return this→id;  
}
```

Χρήση:

```
string proteinName = protein1.getName();  
int proteinId = protein1.getId();
```

Χαρακτηριστικά:

- Return type: ο τύπος του πεδίου
- Ονομασία: συνήθως `getXXX`
- Δεν αλλάζουν την κατάσταση του αντικειμένου

8. Δείκτης `this` ★ ΣΗΜΑΝΤΙΚΟΤΑΤΟ

Ορισμός:

Ειδικός **δείκτης** (pointer) που δείχνει στο **τρέχον αντικείμενο** - δηλαδή το αντικείμενο που κάλεσε τη συγκεκριμένη μέθοδο.

Σύνταξη:

```
this→πεδίο  
this→μέθοδος()
```

Πότε χρειάζεται:

1. Όταν η παράμετρος έχει το ίδιο όνομα με το πεδίο

```
void setId(int id) {  
    this→id = id; // this→id = πεδίο κλάσης  
                // id = παράμετρος συνάρτησης  
}
```

2. Για σαφήνεια και αναγνωσιμότητα

```
void describe() {  
    cout << this→name; // Ξεκάθαρο ότι είναι πεδίο κλάσης  
}
```

3. Όταν θέλουμε να επιστρέψουμε το ίδιο το αντικείμενο

```
Protein& setName(string name) {  
    this→name = name;  
    return *this; // Επιστροφή του αντικειμένου  
}
```

Πώς δουλεύει:

Όταν γράφετε:

```
protein1.setName("PTEN");
```

Η C++ εσωτερικά κάνει κάτι σαν:

```
setName(&protein1, "PTEN"); // Περνάει τη διεύθυνση του protein1  
// Μέσα στη συνάρτηση, this = &protein1
```

Αναλογία:

Φανταστείτε ότι είστε σε ένα δωμάτιο με πολλά άτομα. Το `this` είναι σαν να λέτε:

- "το **δικό μου** όνομα" (αντί για απλά "όνομα")
- "η **δική μου** διεύθυνση" (αντί για απλά "διεύθυνση")

Το `this` σας επιτρέπει να αναφερθείτε στον **εαυτό σας** μέσα στην κλάση.

Παράδειγμα με διαφορά:

```
// ΧΩΡΙΣ this (όταν δεν υπάρχει σύγκρουση ονομάτων)  
void setName(string newName) {  
    name = newName; // Λειτουργεί, αλλά λιγότερο σαφές  
}  
  
// ME this (recommended)  
void setName(string name) {  
    this->name = name; // Ξεκάθαρο ποιο είναι το πεδίο  
}
```



ΒΙΟΛΟΓΙΚΗ ΣΗΜΑΣΙΑ

Πρωτεΐνη (Protein)

Οι πρωτεΐνες είναι μακρομόρια που αποτελούνται από αλυσίδες αμινοξέων.

Πεδία:

- **id:** Μοναδικός αριθμός αναγνώρισης της πρωτεΐνης
- **name:** Όνομα πρωτεΐνης (π.χ. PTEN, TP53, BRCA1)

- **sequence:** Αλληλουχία αμινοξέων - η σειρά των "τουβλάκιων" που φτιάχνουν την πρωτεΐνη
 - Παράδειγμα: `MTAIKEIVSRNKRRYQEDGFDLDTYIYPNIIAMGFPA`
 - Κάθε γράμμα αντιπροσωπεύει ένα αμινοξύ

Γιατί είναι σημαντικές:

- Εκτελούν σχεδόν όλες τις λειτουργίες στον οργανισμό
- Ένζυμα, ορμόνες, αντισώματα - όλα είναι πρωτεΐνες

Γονίδιο (Gene)

Τα γονίδια είναι τμήματα DNA που περιέχουν τις οδηγίες για τη σύνθεση πρωτεϊνών.

Πεδία:

- **id:** Μοναδικός αριθμός αναγνώρισης
- **name:** Όνομα γονιδίου (π.χ. BRCA1, TP53)
- **chrom:** Χρωμόσωμα όπου βρίσκεται το γονίδιο
 - Οι άνθρωποι έχουν 23 ζεύγη χρωμοσωμάτων
 - Παράδειγμα: "17" = χρωμόσωμα 17
- **start:** Θέση έναρξης του γονιδίου στο χρωμόσωμα (αριθμός νουκλεοτιδίου)
- **end:** Θέση τέλους του γονιδίου
- **strand:** Κατεύθυνση ανάγνωσης του DNA
 - `+` = forward strand (θετικός κλώνος)
 - `-` = reverse strand (αρνητικός κλώνος)

Παράδειγμα:

Gene BRCA1 (chr17:43044295-43170245, + strand)

Σημαίνει: Το γονίδιο BRCA1 βρίσκεται στο χρωμόσωμα 17, από τη θέση 43,044,295 έως τη θέση 43,170,245, και διαβάζεται στον θετικό κλώνο.

Γιατί είναι σημαντικά:

- Περιέχουν τις "συνταγές" για τις πρωτεΐνες
- Μεταλλάξεις σε γονίδια μπορεί να προκαλέσουν ασθένειες

ΔΟΜΗ ΠΡΟΓΡΑΜΜΑΤΟΣ C++

Γενική Δομή

```
#include <iostream> // Για input/output
#include <string>    // Για strings
using namespace std;

// 1. Ορισμός κλάσεων
class Protein {
    // ...
};

class Gene {
    // ...
};

// 2. Κύριο πρόγραμμα
int main() {
    // Δημιουργία αντικειμένων
    // Χρήση μεθόδων
    return 0;
}
```

ΠΛΗΡΗΣ ΛΥΣΗ ΜΕ ΣΧΟΛΙΑ

```
#include <iostream>
#include <string>
using namespace std;
```

```

// ΚΛΑΣΗ PROTEIN
class Protein {
private:
    int id;
    string name;
    string sequence;

public:
    // Setters
    void setId(int newId) {
        this->id = newId;
    }

    void setName(string newName) {
        this->name = newName;
    }

    void setSequence(string newSequence) {
        this->sequence = newSequence;
    }

    // Getters
    int getId() {
        return this->id;
    }

    string getName() {
        return this->name;
    }

    string getSequence() {
        return this->sequence;
    }

    // Μέθοδος περιγραφής

```

```

void describe() {
    cout << "protein " << this->name << " "
        << this->sequence << endl;
}

// Προαιρετική μέθοδος μήκους
int length() {
    return this->sequence.length();
}
};

// ΚΛΑΣΗ GENE
class Gene {
private:
    int id;
    string name;
    string chrom;
    int start;
    int end;
    char strand;

public:
    // Setters
    void setId(int newId) {
        this->id = newId;
    }

    void setName(string newName) {
        this->name = newName;
    }

    void setChrom(string newChrom) {
        this->chrom = newChrom;
    }

    void setStart(int newStart) {

```

```
    this→start = newStart;
}

void setEnd(int newEnd) {
    this→end = newEnd;
}

void setStrand(char newStrand) {
    this→strand = newStrand;
}

// Getters
int getId() {
    return this→id;
}

string getName() {
    return this→name;
}

string getChrom() {
    return this→chrom;
}

int getStart() {
    return this→start;
}

int getEnd() {
    return this→end;
}

char getStrand() {
    return this→strand;
}
```

```

// Μέθοδος περιγραφής
void describe() {
    cout << "Gene " << this->name << " (chr"
        << this->chrom << ":" << this->start
        << "-" << this->end << ", "
        << this->strand << " strand)" << endl;
}
};

// ΚΥΡΙΟ ΠΡΟΓΡΑΜΜΑ
int main() {
    // Δημιουργία πρωτεϊνών
    Protein protein1;
    protein1.setId(1);
    protein1.setName("PTEN");
    protein1.setSequence("MTAIKEIVSRNKRRYQEDGFDLTLTYIPNIIAMGFP
A");

    Protein protein2;
    protein2.setId(2);
    protein2.setName("TP53");
    protein2.setSequence("MEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLP
S");

    // Δημιουργία γονιδίων
    Gene gene1;
    gene1.setId(1);
    gene1.setName("BRCA1");
    gene1.setChrom("17");
    gene1.setStart(43044295);
    gene1.setEnd(43170245);
    gene1.setStrand('+');

    Gene gene2;
    gene2.setId(2);
    gene2.setName("TP53");

```

```

gene2.setChrom("17");
gene2.setStart(7668421);
gene2.setEnd(7687490);
gene2.setStrand('-');

// Εκτύπωση
cout << "=== ΠΡΩΤΕΪΝΕΣ ===" << endl;
protein1.describe();
cout << "Μήκος: " << protein1.length() << endl << endl;

protein2.describe();
cout << "Μήκος: " << protein2.length() << endl << endl;

cout << "=== ΓΟΝΙΔΙΑ ===" << endl;
gene1.describe();
gene2.describe();

return 0;
}

```

✓ ΣΗΜΕΙΑ ΠΡΟΣΟΧΗΣ

1. Private vs Public

- Τα **πεδία** είναι `private` (προστασία)
- Οι **μέθοδοι** είναι `public` (πρόσβαση)

2. Χρήση του `this->`

- Χρησιμοποιείται για να διακρίνουμε πεδία από παραμέτρους
- Κάνει τον κώδικα πιο σαφή και αναγνώσιμο

3. Return Types

- **Setters:** `void` (δεν επιστρέφουν τίποτα)

- **Getters:** τύπος του πεδίου που διαβάζουν

4. Απαραίτητα includes

```
#include <iostream> // Για cout, cin
#include <string>    // Για string
using namespace std; // Για να μην γράφουμε std::
```

ΑΝΑΜΕΝΟΜΕΝΟ ΑΠΟΤΕΛΕΣΜΑ

```
=== ΠΡΩΤΕΪΝΕΣ ===
protein PTEN MTAIKEIVSRNKRRYQEDGFDLDTYIYPNIIAMGFPA
Μήκος: 40

protein TP53 MEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLPS
Μήκος: 38

=== ΓΟΝΙΔΙΑ ===
Gene BRCA1 (chr17:43044295-43170245, + strand)
Gene TP53 (chr17:7668421-7687490, - strand)
```

ΣΥΝΟΨΗ ΜΑΘΗΣΙΑΚΩΝ ΣΤΟΧΩΝ

Μετά από αυτό το εργαστήριο, θα έχετε κατανοήσει:

1.  Πώς να ορίζετε **κλάσεις** στη C++
2.  Τη διαφορά μεταξύ **πεδίων** και **μεθόδων**
3.  Την έννοια του **encapsulation** (εγκλωβισμός)
4.  Πώς λειτουργούν τα **setters** και **getters**
5.  Τι είναι ο δείκτης **this** και πότε τον χρησιμοποιούμε
6.  Πώς να δημιουργείτε και να χρησιμοποιείτε **αντικείμενα**
7.  Πώς να αναπαριστάτε **πραγματικές βιολογικές οντότητες** με κλάσεις

➡ ΕΠΟΜΕΝΑ ΒΗΜΑΤΑ

Στο επόμενο εργαστήριο θα μάθετε:

- **Constructors:** Πώς αρχικοποιούνται αυτόματα τα αντικείμενα
- **Destructors:** Τι συμβαίνει όταν καταστρέφονται τα αντικείμενα
- **Διαχείριση μνήμης:** Πώς η C++ διαχειρίζεται τα αντικείμενα

💡 ΕΠΙΠΛΕΟΝ TIPS

Καλές Πρακτικές:

1. Πάντα χρησιμοποιείτε `this→` για σαφήνεια
2. Κάντε τα πεδία `private`
3. Δώστε περιγραφικά ονόματα σε μεθόδους και πεδία
4. Προσθέστε σχόλια όπου χρειάζεται

Κοινά Λάθη:

1. ❌ Ξεχνάτε το `;` μετά το `}` της κλάσης
2. ❌ Βάζετε `()` στη δημιουργία αντικειμένου: `Protein p();` ❌
3. ❌ Προσπαθείτε να προσπελάσετε `private` πεδία απευθείας
4. ❌ Ξεχνάτε το `return` στους getters

Σωστή Σύνταξη:

```
Protein p;    // ✅ Σωστό  
Protein p();  // ❌ Λάθος (δήλωση συνάρτησης)
```

Καλή Επιτυχία στο Εργαστήριο! 🚀